Petra Chinsangaram

Mar 11, 2026

Foundations of Python Programming

Assignment 07

https://github.com/petra-cc/IntroToProg-Python-Mod07

Assignment 7: Classes and Objects

Introduction

For this assignment, I built upon the previous assignment by creating data classes with which I could create objects to categorize my data, rather than using dictionaries. I started by reading and watching the module 7 resources. I then created my program.

Creating the Program

First, I opened the assignment 7 starter Python script in PyCharm. I updated the change log with my name and the date, as well as the change I was making. The constants and variables were the same as in the previous assignment, so I didn't have to make any changes there.

Creating Data Classes: Person

My first task is to create my parent data class, which is the Person class (Figure 1). First, I build the constructor using the __init__ method. The constructor will be automatically called when an instance of the Person class is created, and will be used to set the attributes of that instance. The first parameter is self, with refers to an instance of the Person class when it is called. I also have the parameters first_name and last_name, which are given default values of empty strings. I then set the first_name and last_name attributes of an instance to take the arguments provided for the first_name and last_name parameters.

I then create getter and setter methods for each of my attributes. In the case of first_name, the getter is preceded by the @property decorator and simply returns the first name attribute of an instance, set to title case. The setter is preceded by the @first_name.setter decorator and is a bit more complicated. Along with self (which refers back to the specific object instance being called), the setter has a value parameter, which will be filled either by the user entering data or the program reading from a file. An if/else block checks if the value is alphabetic or an empty string, in which case the first_name attribute is set to value. Otherwise, a ValueError with a personalized message is provided, which will be handled later in the input_student_data method. The last name getter and setter are set up in the same way.

The last method in this class is the __str__ method. This overrides the default string called by Python when the an object is called, which indicates the memory address of the object. Instead, I set the method to return a comma-separated string containing the first and last name.

```python
class Person:

    # Constructor
    def __init__(self, first_name: str = "", last_name: str = ""):
        self.first_name = first_name
        self.last_name = last_name

    # First name getter
    @property
    def first_name(self):
        return self.__first_name.title

    # First name setter
    @first_name.setter
    def first_name(self, value: str):
        if value.isalpha() or value == "":
            self.__first_name = value
        else:
            raise ValueError("The first name should contain only letters.")

    # Last name getter
    @property
    def last_name(self):
        return self.__last_name.title

    # Last name setter
    @last_name.setter
    def last_name(self, value: str):
        if value.isalpha() or value == "":
            self.__last_name = value
        else:
            raise ValueError("The last name should contain only letters.")
```

```
    # Override __str__ method
    def __str__(self):
        return f"{self.first_name},{self.last_name}"
```

Figure 1: Person class with methods. Descriptive document strings removed to shorten this document.

Creating Data Classes: Student(Person)

The other data class I create is the Student class, which is a child of the Person class (Figure 2). This means that the Student class can inherit properties and methods from the Person class. For example, since a Student has a first and last name like a Person, I can use super().__init__ in my Student constructor to use the first_name and last_name properties from the Person class. Additionally, a Student requires a course name property, so I add that separately.

Since Student is inheriting first_name and last_name from Person, I only need to create a getter and setter for course_name and a new __str__ method to include the course name in the default string.

```
class Student(Person):


    # Constructor
    def __init__(self, first_name: str = "",
                 last_name: str = "", course_name: str = ""):
        super().__init__(first_name=first_name,
                         last_name=last_name)
        self.course_name = course_name


    # Course name getter
    @property
    def course_name(self):
        return self.__course_name.title


    # Course name setter
    @course_name.setter
    def course_name(self, value: str):
        self.__course_name = value
```

```
    # Override __str__ method
    def __str__(self):
        return (f"{self.first_name},"
                f"{self.last_name},{self.course_name}")
```

Figure 2: Student class with methods. Descriptive document strings removed to shorten this document.

JSON Conversion: Reading from File

Since the data is no longer being processed as a list of dictionaries, it becomes a bit more complicated to read from and write to a JSON file. To read from the JSON file, I begin as normal, opening the file in read mode and loading the data into a variable. However, now I have to convert this data to a list of Student objects. I start a for loop so I can load multiple students. For each student, I create a new instance of my Student class, assigning the first name, last name, and course name from the dictionary to the Student attributes. I save this object to a temporary variable and append it to my student_data variable, creating a list of Student objects. I can then close the file (Figure 3).

```
# Get a list of dictionary rows from the data file
file = open(file_name, "r")
json_students = json.load(file)

# Convert the list of dictionary rows into a list of Student objects
for student in json_students:
    student_object: Student = Student(first_name=
                                      student["FirstName"],
                                      last_name=
                                      student["LastName"],
                                      course_name=
                                      student["CourseName"])
    student_data.append(student_object)
file.close()
```

Figure 3: Code reading JSON data from file and converting to a list of Student objects

JSON Conversion: Writing to File

To write to a JSON file, I have to do the reverse: convert a list of Student objects to a list of dictionaries. I open the file in write mode, then create a list of json students. I then use a for loop to build a list of dictionaries by calling each attribute of the Student object before appending the dictionary to my list. Now having a list of dictionaries, I can write it to the file and close the file.

Initially, I had an issue with my code referring to the correct string when calling to attributes, such as in this section. However, I was able to fix this by adding the empty parenthesis after calling each attribute.

```
file = open(file_name, "w")


json_students: list = []
for student in student_data:
   student_json: dict = {"FirstName": student.first_name(),
                         "LastName": student.last_name(),
                         "CourseName": student.course_name()}
   json_students.append(student_json)


json.dump(json_students, file, indent=2)
file.close()
```

Figure 4: Code converting list of Student objects to list of dictionaries and writing to JSON file

Output Student Data

I update the code for presenting the data to the user by calling my Student attributes rather than dictionary values.

```
for student in student_data:
   print(f'Student {student.first_name()} '
         f'{student.last_name()} '
         f'is enrolled in {student.course_name()}')
```

Figure 5: Updated code to output student data to user

Calling the Functions

I update the code for collecting student data from the user by calling my Student attributes and adding the user's input data. I append this to the student_data variable and print the new student's information to the user.

As in the previous assignment, this is contained within a try/except block which handles a value error (or other exception error) being raised. However, rather than the value error being raised within this function, it will now be raised as part of the validation code in my Student class.

```
student = Student()


student.first_name = input("Enter the student's first name: ")
student.last_name = input("Enter the student's last name: ")
student.course_name = input("Please enter the name of the course: ")


student_data.append(student)
```

```
print()
print(f"You have registered {student.first_name()} "
     f"{student.last_name()} for {student.course_name()}.")
```

Figure 6: Updated code to collect student data input from user

Testing the Program

I tested my program in PyCharm (Figures 7-10) and the Command Prompt (Figures 11-14). Both functioned properly, resulting in an updated JSON file (Figure 15).

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 1
Enter the student's first name: John
Enter the student's last name: Doe
Please enter the name of the course: Biology 101

You have registered John Doe for Biology 101.
```

```
Enter your menu choice number: 2
--------------------------------------------------
Student Vic Vu is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student John Doe is enrolled in Biology 101
--------------------------------------------------
```

```
Enter your menu choice number: 3
-------------------------------------------------
Student Vic Vu is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student John Doe is enrolled in Biology 101
-------------------------------------------------
```

```
Enter your menu choice number: 4
Program Ended
```

Figures 7-10: Program output in PyCharm

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 1
Enter the student's first name: John
Enter the student's last name: Doe
Please enter the name of the course: Biology 101

You have registered John Doe for Biology 101.
```

```
Enter your menu choice number: 2
-------------------------------------------------
Student Vic Vu is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student John Doe is enrolled in Biology 101
-------------------------------------------------
```

```
Enter your menu choice number: 3
-----------------------------------------------
Student Vic Vu is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student John Doe is enrolled in Biology 101
-----------------------------------------------
```

```
Enter your menu choice number: 4
Program Ended
```

Figures 11-14: Program output in Command Prompt


```
[
  {
    "FirstName": "Vic",
    "LastName": "Vu",
    "CourseName": "Python 100"
  },
  {
    "FirstName": "Sue",
    "LastName": "Jones",
    "CourseName": "Python 100"
  },
  {
    "FirstName": "John",
    "LastName": "Doe",
    "CourseName": "Biology 101"
  }
]
```


Figure 15: Resulting Enrollments.json file viewed in PyCharm

Summary

This assignment functions the same as the previous assignment, but is different in the inclusion of data classes. Although this does seem a bit more complex for a simple program like this one, I can see the use

of having classes with shared attributes for a larger-scale version of this registration program. It was interesting to explore code inheritance as well, with the parent and child classes.